

《神经网络与深度学习》

第四章-前馈神经网络

人工智能学院

杨溪

➤神经网络

- 人工神经元
- 网络结构

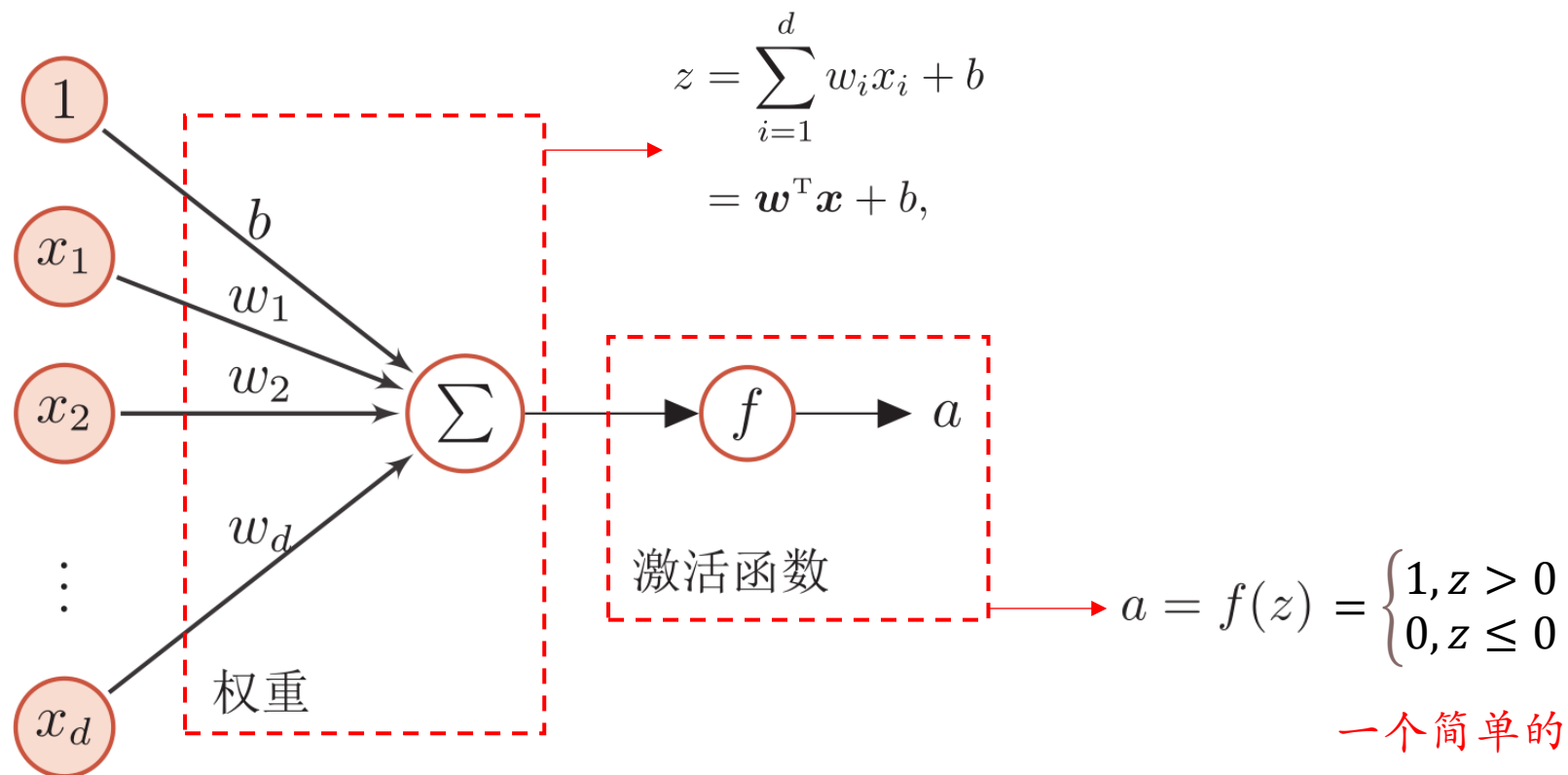
➤前馈神经网络

- 参数学习
- 反向传播算法
- 计算图与自动微分

➤优化问题

人工神经元

◆ 人工神经元，简称神经元，是构成神经网络的基本单元，其主要是模拟生物神经元的结构和特性，接收一组输入信号并产生输出。



一个简单的线性模型！（感知器）

◆ 激活函数应具备以下特点：

◆ 连续并可导（允许少数点上不可导）的非线性函数

◆ 可导的激活函数可以直接利用数值优化的方法来学习网络参数。

◆ 激活函数及其导函数要尽可能的简单

◆ 有利于提高网络计算效率。

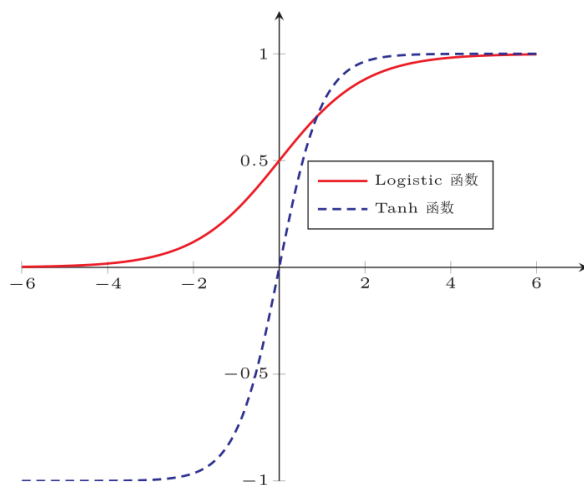
◆ 激活函数的导函数的值域要在一个合适的区间内

◆ 不能太大也不能太小，否则会影响训练的效率和稳定性。

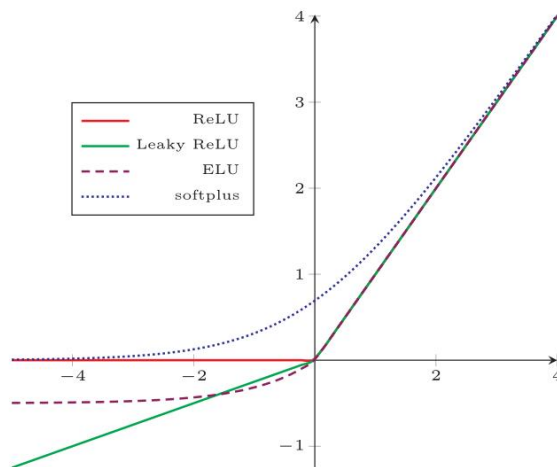
◆ 单调递增 （但并不绝对）

$$a \uparrow = f(z \uparrow)$$

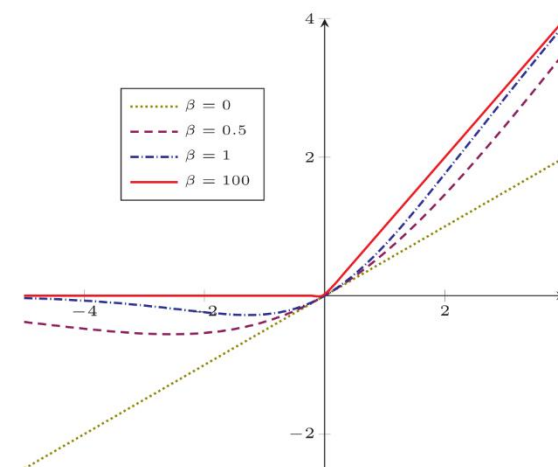
常用的激活函数



S型函数



斜坡函数



复合函数

常见激活函数-Sigmoid型

Logistic函数 $\sigma(x) = \frac{1}{1 + \exp(-x)}$

$\epsilon(0,1)$

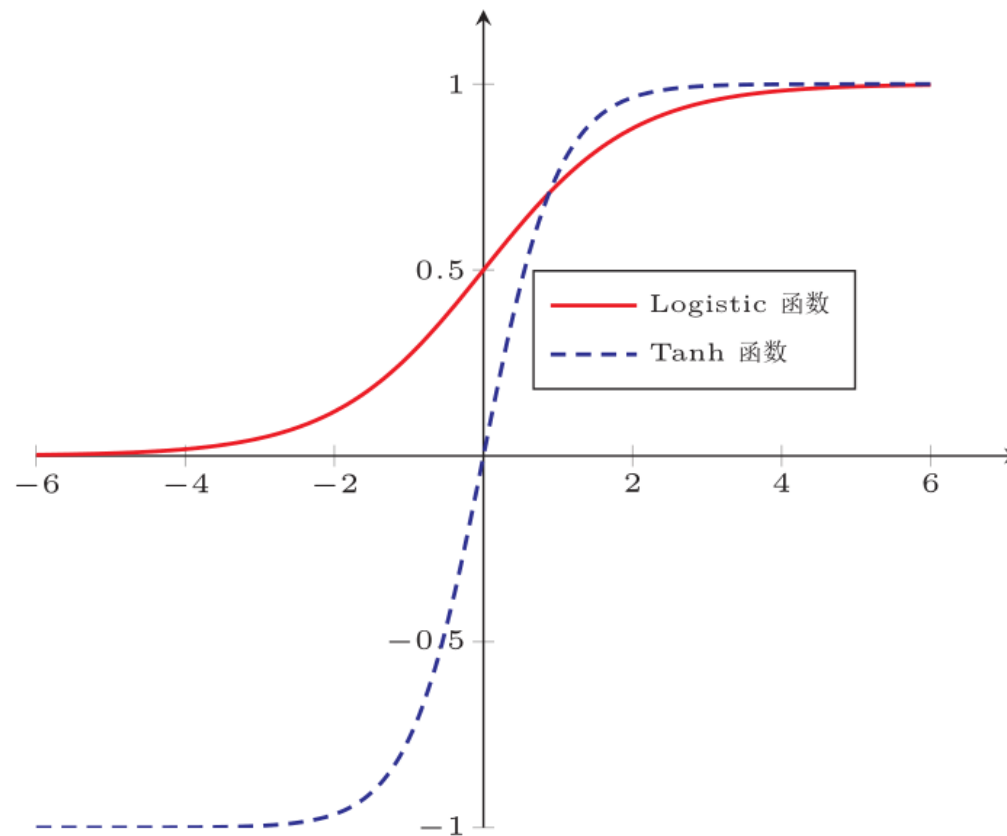
Tanh函数 $\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$

$\epsilon(-1,1)$

$$\tanh(x) = 2\sigma(2x) - 1$$

性质：

- ◆ 饱和函数
- ◆ Tanh函数是零中心化；
Logistic函数的输出恒大于0。



非零中心化的输出会使得其下一层的神经元的输入发生偏置偏移 (bias shift)，并进一步使得梯度下降的收敛速度变慢。

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$
$$= \max(0, x).$$

死亡ReLU问题
(Dying ReLU Problem)

- 数据归一化
- 参数初始化

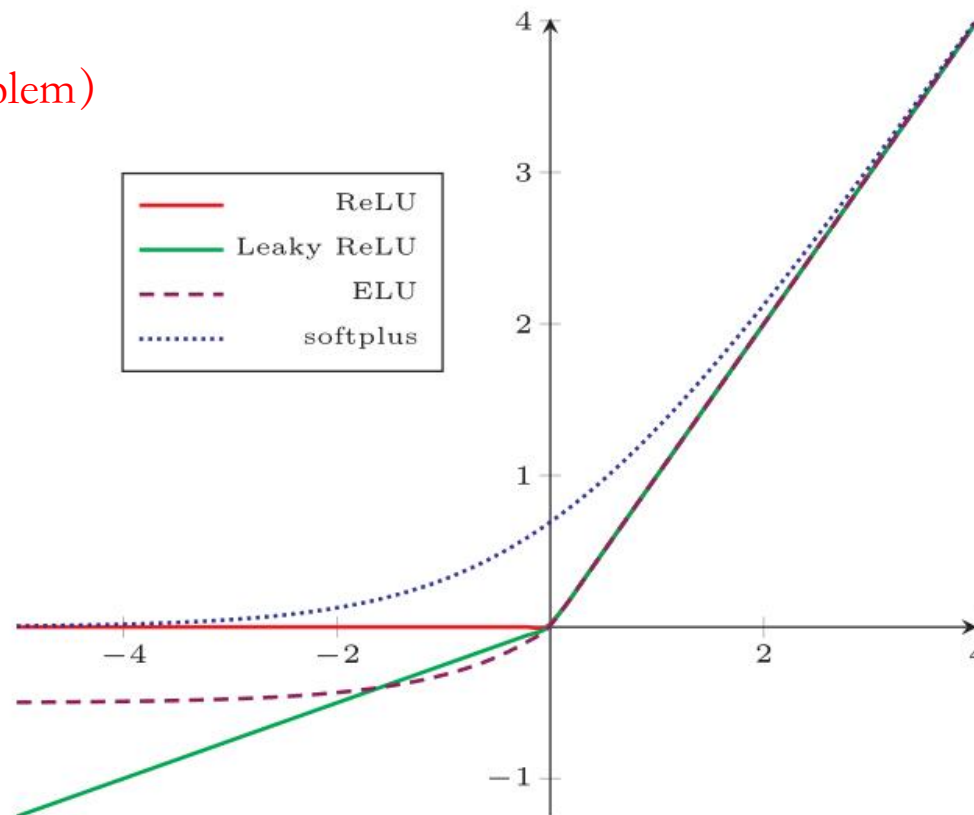
$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \gamma \min(0, x)$$

近似的零中心化的非线性函数

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$
$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

Rectifier函数的平滑版本

$$\text{softplus}(x) = \log(1 + \exp(x))$$



优势:

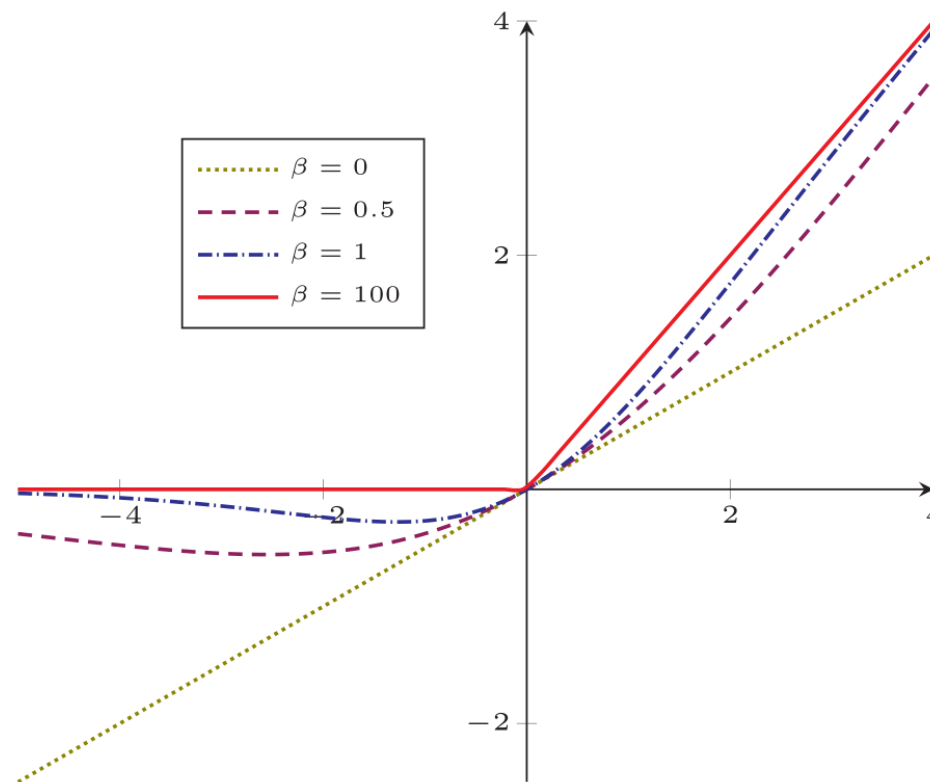
- ◆ 计算上更加高效
- ◆ 生物学合理性 (单侧抑制、宽兴奋边界)
- ◆ 在一定程度上缓解梯度消失问题

Swish函数：一种自门控（Self-Gated）激活函数

Rectifier型 Sigmoid型

$$\text{swish}(x) = x\sigma(\beta x)$$

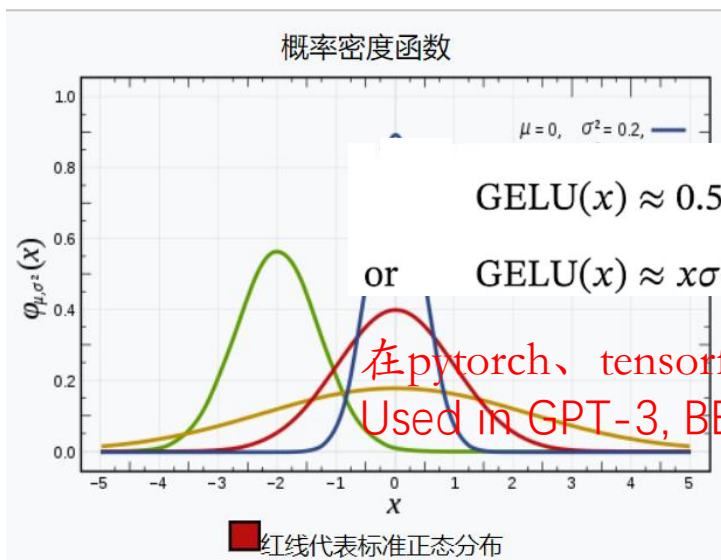
Logistic函数 $\epsilon(0,1)$



◆ 高斯误差线性单元 (Gaussian Error Linear Unit, GELU)

$$\text{GELU}(x) = xP(X \leq x)$$

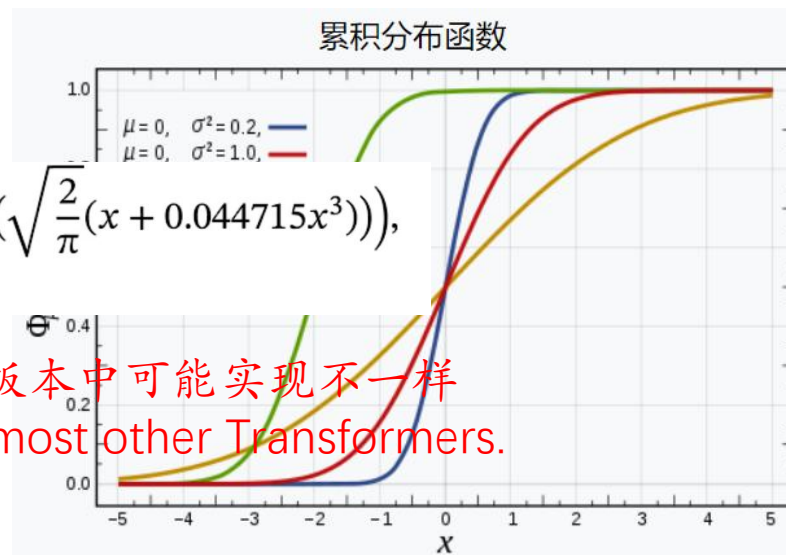
- ◆ 其中 $P(X \leq x)$ 是高斯分布 $N(\mu, \sigma^2)$ 的累积分布函数，其中 μ, σ 为超参数，一般设 $\mu = 0, \sigma = 1$ 即可
- ◆ 由于高斯分布的累积分布函数为Sigmoid型函数，因此GELU可以用Tanh函数或Logistic函数来近似



$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right),$$

$$\text{or } \text{GELU}(x) \approx x\sigma(1.702x).$$

在pytorch、tensorflow不同版本中可能实现不一样
Used in GPT-3, BERT, and most other Transformers.



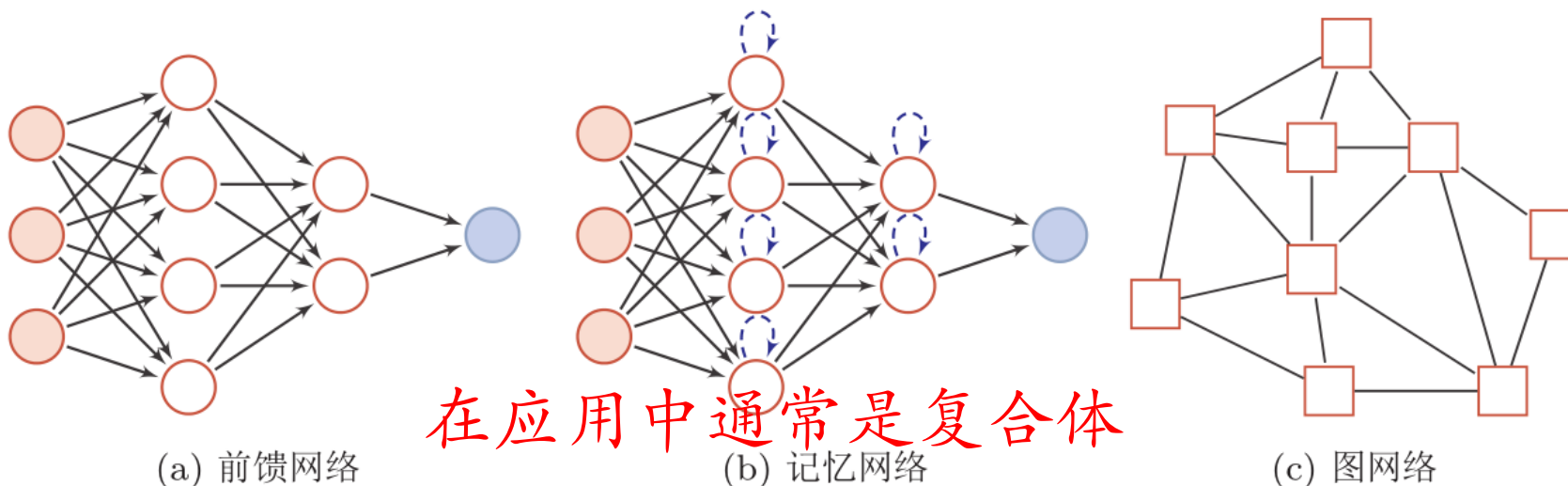
S型

激活函数	函数	导数
Logistic 函数	$f(x) = \frac{1}{1+\exp(-x)}$	$f'(x) = f(x)(1 - f(x))$
Tanh 函数	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$f'(x) = 1 - f(x)^2$
ReLU 函数	$f(x) = \max(0, x)$	$f'(x) = I(x > 0)$
ELU 函数	$f(x) = \max(0, x) + \min(0, \gamma(\exp(x) - 1))$	$f'(x) = I(x > 0) + I(x \leq 0) \cdot \gamma \exp(x)$
SoftPlus 函数	$f(x) = \log(1 + \exp(x))$	$f'(x) = \frac{1}{1+\exp(-x)}$

神经网络

◆ 人工神经网络主要由大量的神经元以及它们之间的有向连接构成。主要考虑三个方面：

1. 神经元的激活规则，指神经元输入到输出之间的映射关系，一般使用非线性函数；
2. 网络的拓扑结构，即不同神经元之间的连接关系；
3. 学习算法，通过在已有数据上训练，得到神经网络中的参数。



在应用中通常是复合体

圆形节点表示一个神经元，方形节点表示一组神经元。

◆神经网络最早作为一种主要的连接主义模型。20世纪80年代后期，最流行的一种连接主义模型是分布式并行处理（Parallel Distributed Processing, PDP）网络。

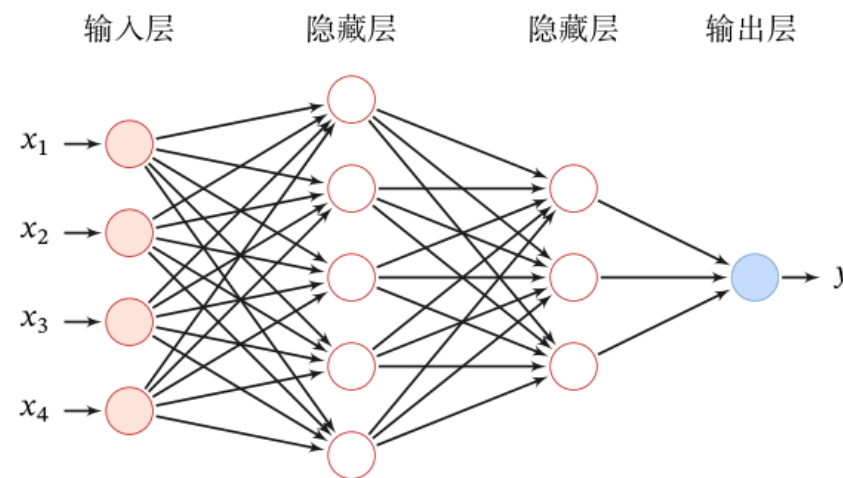
◆分布式并行处理网络的三个特性：

1. 信息表示是分布式的（非局部的）；
2. 记忆和知识是存储在单元之间的连接上；
3. 通过逐渐改变单元之间的连接强度来学习新的知识。

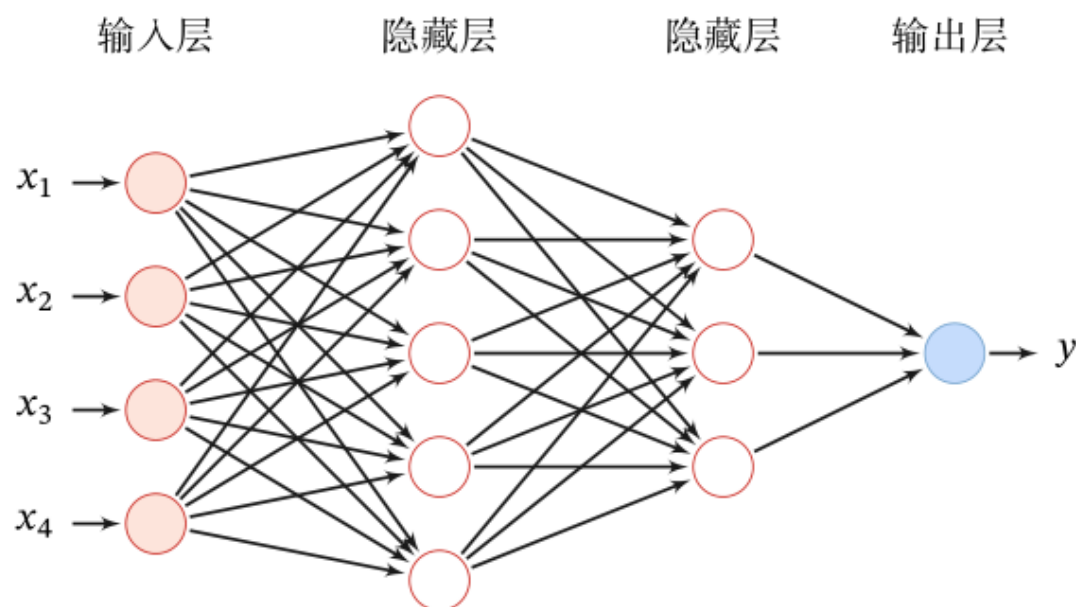
◆在引入**误差反向传播**来改进其学习能力之后，神经网络也越来越多地应用在各种机器学习任务上。

前馈神经网络

- ◆也可称为全连接神经网络、多层感知器;
- ◆整个网络中无反馈, 信号从输入层向输出层单向传播, 可以用一个有向无环图表示;
- ◆各神经元分别属于不同的层, 层内无连接;
- ◆相邻两层之间的神经元全部两两连接;



◆ 给定一个前馈神经网络，用下面图示和记号来描述网络结构：



超参数

参数

活性值

记号	含义
L	神经网络的层数
M_l	第 l 层神经元的个数
$f_l(\cdot)$	第 l 层神经元的激活函数
$\mathbf{W}^{(l)} \in \mathbb{R}^{M_l \times M_{l-1}}$	第 $l-1$ 层到第 l 层的权重矩阵
$\mathbf{b}^{(l)} \in \mathbb{R}^{M_l}$	第 $l-1$ 层到第 l 层的偏置
$\mathbf{z}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的净输入 (净活性值)
$\mathbf{a}^{(l)} \in \mathbb{R}^{M_l}$	第 l 层神经元的输出 (活性值)

◆ 前馈神经网络按照下面公式进行信息传播。

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)},$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)}).$$

◆ 前馈计算

根据上述公式，可以将前向计算过程抽象为下面式子：

$$\mathbf{x} = \mathbf{a}^{(0)} \xrightarrow{w^1} \mathbf{z}^{(1)} \xrightarrow{f_1(\cdot)} \mathbf{a}^{(1)} \rightarrow \mathbf{z}^{(2)} \rightarrow \dots \rightarrow \mathbf{a}^{(L-1)} \rightarrow \mathbf{z}^{(L)} \rightarrow \mathbf{a}^{(L)} = \phi(\mathbf{x}; \mathbf{W}, \mathbf{b})$$

输入

定理：令 $\varphi(\cdot)$ 是一个非常数、有界、单调递增的连续函数， $\mathcal{I}_D$ 是一个D维的单位超立方体 $[0, 1]^D$ ， $C(\mathcal{I}_D)$ 是定义在 $\mathcal{I}_D$ 上的连续函数集合。对于任意给定的一个函数 $f \in C(\mathcal{I}_D)$ ，存在一个整数 M 和一组实数 $v_m, b_m \in \mathbb{R}$ ，以及实数向量 $\mathbf{w}_m \in \mathbb{R}^D, m = 1, \dots, M$ ，以至于我们可以定义函数

$$F(x) = \sum_{m=1}^M v_m \varphi(\omega_m^T x + b_m),$$

作为函数 f 的近似实现，即

$$|F(x) - f(x)| < \varepsilon, \forall x \in \mathcal{I}_D,$$

其中 ε 是一个很小的正数。

- ◆ 根据通用近似定理，对于具有线性输出层和至少一个使用“挤压”性质的激活函数的隐藏层组成的前馈神经网络，只要其隐藏层神经元的数量足够，它可以以任意的精度来近似任何从一个定义在实数空间中的有界闭集函数。
- ◆ 因此神经网络可以作为一个“万能”函数来使用，用来进行复杂的特征转换或逼近一个复杂的条件分布。

$$\hat{y} = g(\varphi(\mathbf{x}), \theta)$$

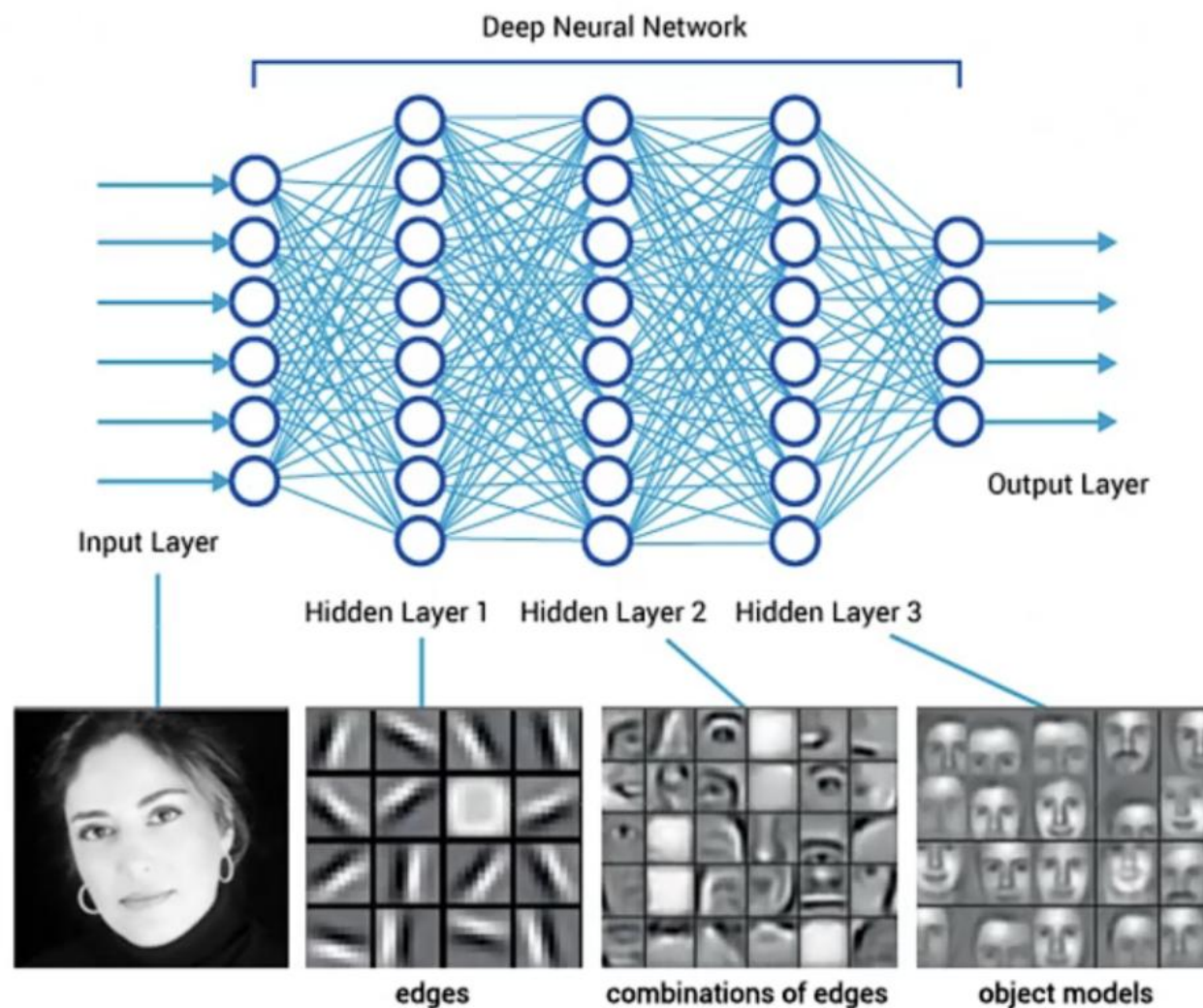
分类器，
有时可认为是神经网络的最后一层

神经网络

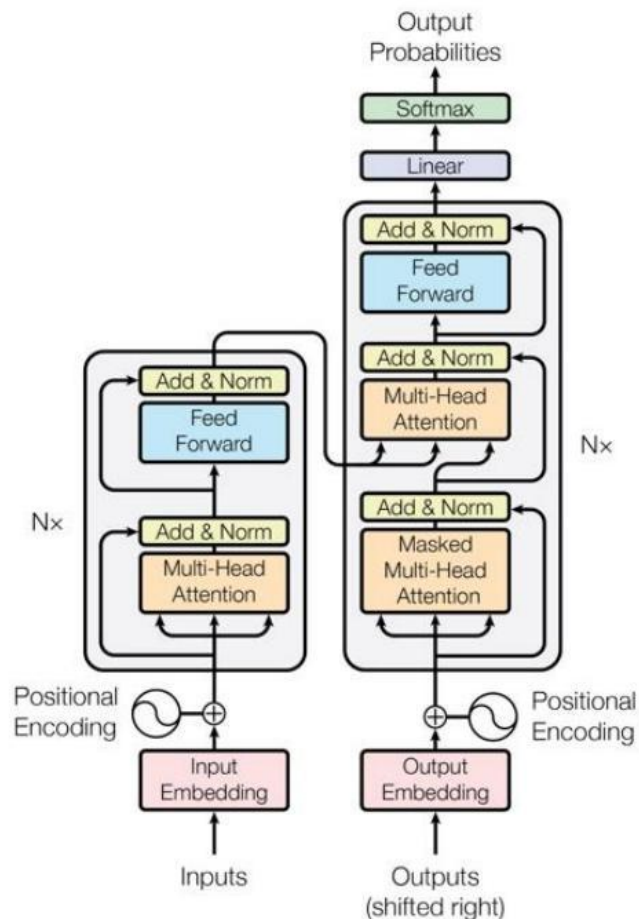
其中 $g(\cdot)$ 为线性或非线性的分类器， θ 为分类器 $g(\cdot)$ 的参数， $\mathbf{x}$ 为样本的输入， $\hat{y}$ 为分类器的输出。

例：深层前馈神经网络

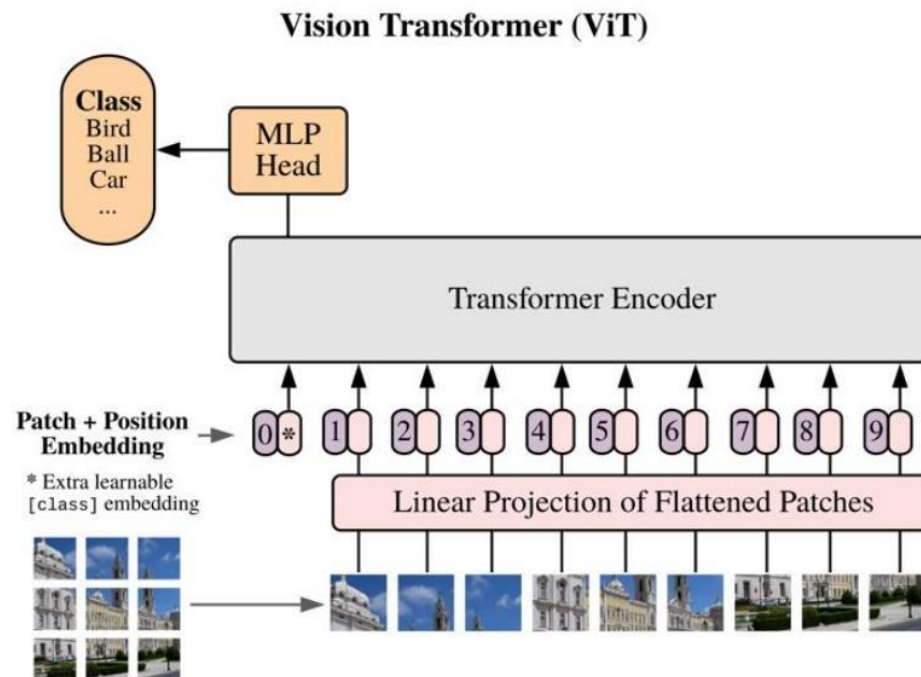
图像
分类



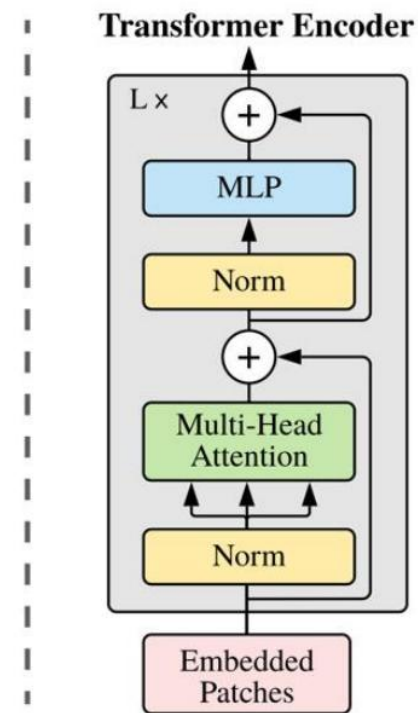
现代大模型中的前馈层



Vaswani et al, "Attention is all you need",
NeurIPS 2017



Dosovitskiy et al, "An Image is Worth 16x16 Words:
Transformers for Image Recognition at Scale", ICLR 2021



Article

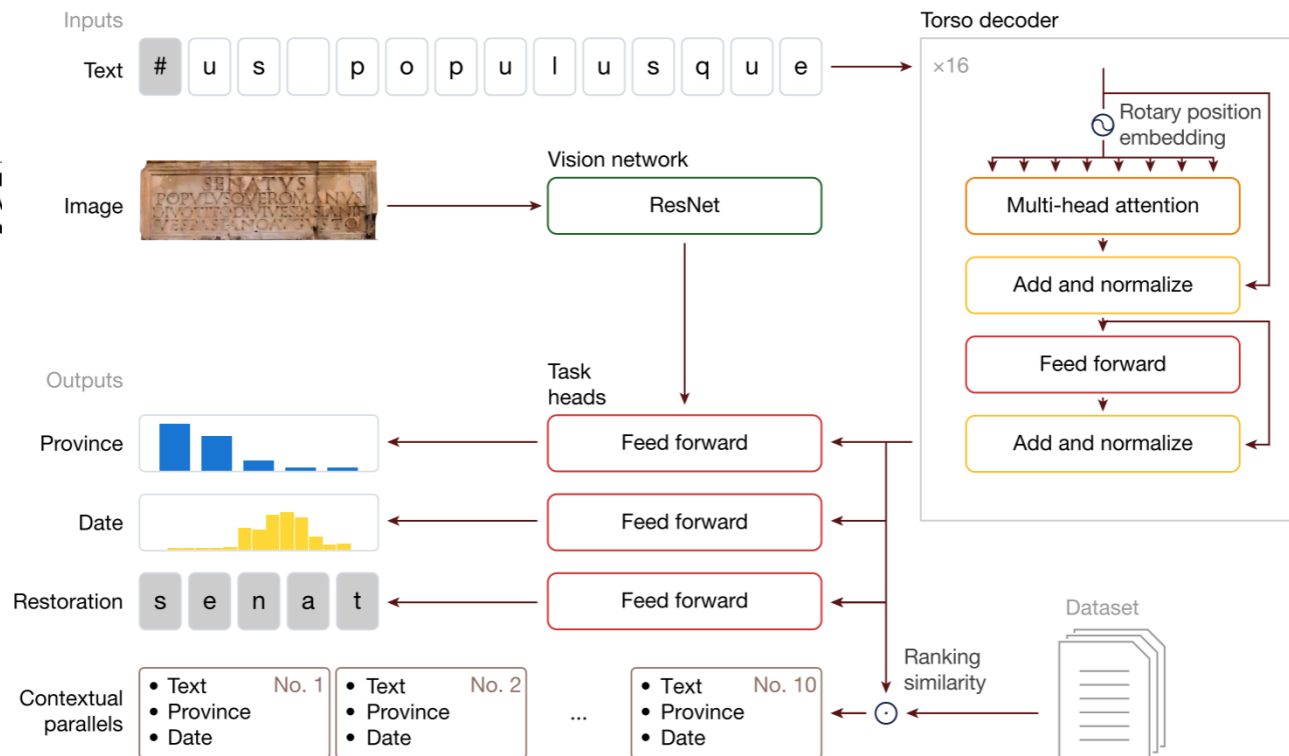
Contextualizing ancient texts with generative neural networks

<https://doi.org/10.1038/s41586-025-09292-5>

Received: 25 December 2024

Accepted: 16 June 2025

Yannis Assael^{1,8}, Thea Sommerschild^{2,8}, Alison Cooley³, Brendan Shillingford⁴, John Pavlopoulos⁴, Priyanka Suresh¹, Bailey Herms⁵, Justin Grayston⁵, Benjamin H. Nicholas Dietrich¹, Robbe Wulgaert⁶, Jonathan Prag⁷, Alex Mullen² & Shakir Moha



参数学习

对于多分类问题

◆如果使用Softmax回归分类器，相当于网络最后一层设置C 个神经元，其输出经过Softmax函数进行归一化后可以作为每个类的条件概率。

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(L)})$$

◆采用交叉熵损失函数，对于样本 $(\mathbf{x}, \mathbf{y})$ ，其损失函数为

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = -\mathbf{y}^T \log \hat{\mathbf{y}}$$

◆ 给定训练集为 $D = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$ ，将每个样本 $\mathbf{x}^{(n)}$ 输入给前馈神经网络，得到网络输出为 $\hat{y}^{(n)}$ ，其在数据集 D 上的结构化风险函数为：

$$\mathcal{R}(W, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, \hat{y}^{(n)}) + \frac{1}{2} \lambda \boxed{\|W\|_F^2}$$

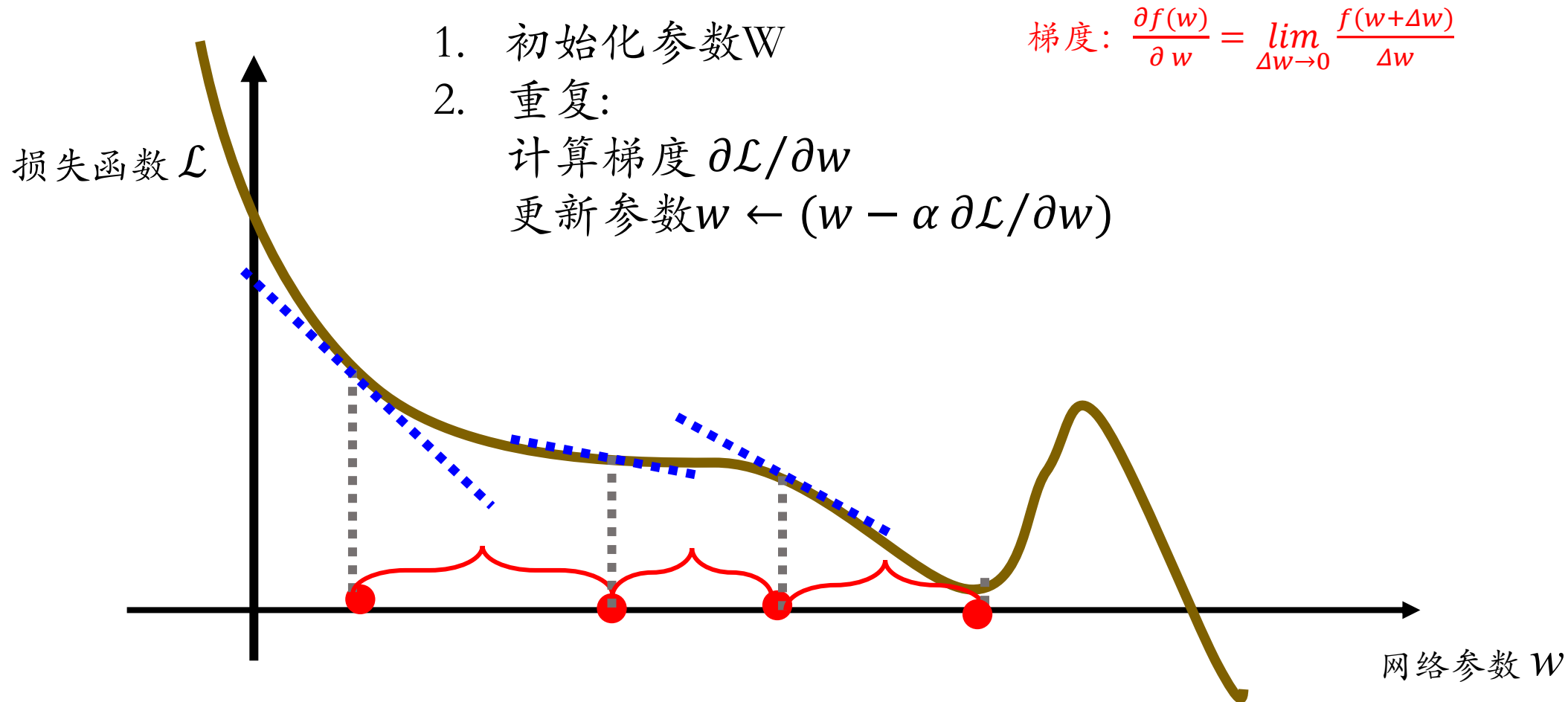
防止过拟合的正则化项

◆ 通过梯度下降更新参数：

$$W^{(l)} \leftarrow W^{(l)} - \overset{\text{学习率}}{\alpha} \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial W^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \frac{\partial \mathcal{R}(W, \mathbf{b})}{\partial \mathbf{b}^{(l)}}$$

梯度下降



神经网络为一个复杂的复合函数

◆ 链式法则

$$\hat{y} = f^5(f^4(f^3(f^2(f^1(x)))))) \rightarrow \frac{\partial \hat{y}}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$

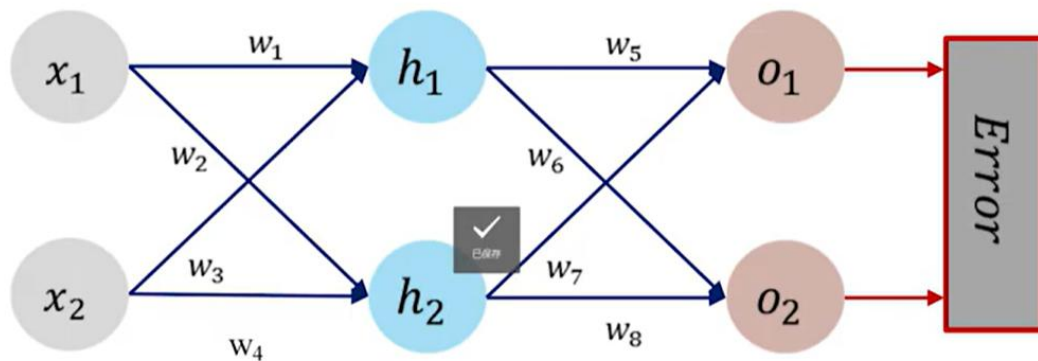
◆ 反向传播算法：根据前馈网络的特点而设计的高效方法

随着网络复杂化，计算变得复杂

◆ 更通用的计算方法

自动微分 (Automatic Differentiation, AD)

反向传播算法



$$Error = \frac{1}{2} \sum_{i=1}^2 (o_i - y_i)^2$$

1. 计算 $w_5 \sim w_8$ 的梯度 (以 w_5 为例)

$$\delta_5 = \frac{\partial Error}{\partial w_5} = \frac{\partial Error}{\partial o_1} * \frac{\partial o_1}{\partial \ln o_1} * \frac{\partial \ln o_1}{\partial w_5}$$

where,

$$\frac{\partial Error}{\partial o_1} = o_1 - y_1 \quad \leftarrow Error = \frac{1}{2} \sum_{i=1}^2 (o_i - y_i)^2$$

$$\frac{\partial o_1}{\partial \ln o_1} = o_1 * (1 - o_1) \quad \leftarrow o_1 = Out_{o_1} = Sigmoid(\ln o_1)$$

$$\frac{\partial \ln o_1}{\partial w_5} = h_1 \quad \leftarrow \ln o_1 = w_5 * h_1 + w_7 * h_2$$

机器学习、其他资料

◆ 矩阵微积分 (Matrix Calculus) 是多元微积分的一种表达方式，即使用矩阵和向量来表示因变量每个成分关于自变量每个成分的偏导数。

$$y = f(x) \in R$$

◆ 分母布局 (列向量)

◆ 标量关于向量的偏导数：

$$\frac{\partial y}{\partial \mathbf{x}} = \left[\frac{\partial y}{\partial x_1}, \dots, \frac{\partial y}{\partial x_p} \right]^T$$

◆ 向量关于向量的偏导数：

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \dots & \frac{\partial y_q}{\partial x_1} \\ \vdots & \vdots & \vdots \\ \frac{\partial y_1}{\partial x_p} & \dots & \frac{\partial y_q}{\partial x_p} \end{bmatrix} \in \mathbb{R}^{p \times q}$$

◆ 三种情况:

标量

向量

向量

(1) 若 $x \in \mathbb{R}, \mathbf{y} = g(x) \in \mathbb{R}^M, \mathbf{z} = f(\mathbf{y}) \in \mathbb{R}^N$, 则

$$\frac{\partial \mathbf{z}}{\partial x} = \frac{\partial \mathbf{y}}{\partial x} \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \in \mathbb{R}^{1 \times N}.$$

$1 \times M$

$M \times N$

(2) 若 $\mathbf{x} \in \mathbb{R}^M, \mathbf{y} = g(\mathbf{x}) \in \mathbb{R}^K, \mathbf{z} = f(\mathbf{y}) \in \mathbb{R}^N$, 则

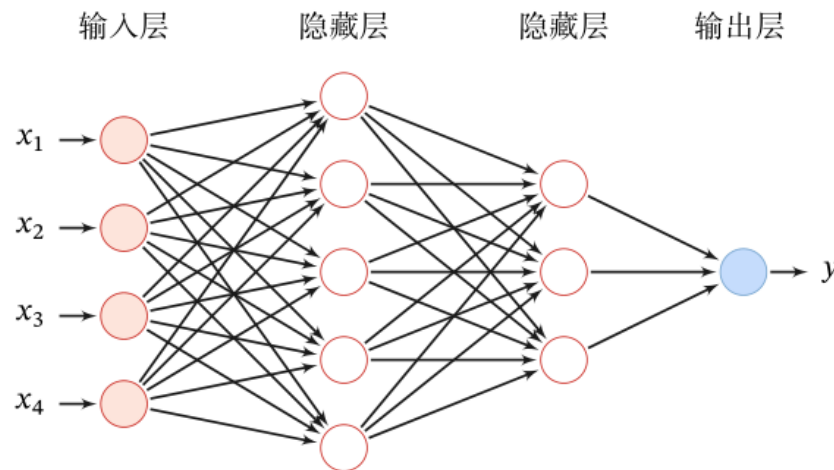
$$\frac{\partial \mathbf{z}}{\partial \mathbf{x}} = \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \frac{\partial \mathbf{z}}{\partial \mathbf{y}} \in \mathbb{R}^{M \times N}.$$

(3) 若 $\mathbf{X} \in \mathbb{R}^{M \times N}$ 为矩阵, $\mathbf{y} = g(\mathbf{X}) \in \mathbb{R}^K, z = f(\mathbf{y}) \in \mathbb{R}$, 则

$$\frac{\partial z}{\partial x_{ij}} = \frac{\partial \mathbf{y}}{\partial x_{ij}} \frac{\partial z}{\partial \mathbf{y}} \in \mathbb{R}.$$

展开, 对矩阵每个元素的偏导, 再拼成矩阵

$$\mathcal{R}(\mathbf{W}, \mathbf{b}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)}) + \frac{1}{2} \lambda \|\mathbf{W}\|_F^2$$



对第 l 层:

$$\frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{W}^{(l)}} \quad \rightarrow \quad \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} \qquad \frac{\partial \mathcal{R}(\mathbf{W}, \mathbf{b})}{\partial \mathbf{b}^{(l)}} \quad \rightarrow \quad \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial b^{(l)}}$$

- ◆在对第 l 层中的参数和偏置项计算偏导数时，一种简便的方法是，先计算损失函数关于参数矩阵中每个元素的偏导数，根据链式法则可以得到：

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\begin{aligned} \mathbf{z}^{(l)} &= \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \\ \mathbf{a}^{(l)} &= f_l(\mathbf{z}^{(l)}). \end{aligned}$$

- ◆两个公式右侧的第二项是相同的，因此分别计算三个偏导数即可。

计算对参数的偏导数：

$$\begin{aligned}\frac{\partial \mathbf{z}^{(l)}}{\partial w_{ij}^{(l)}} &= \left[\frac{\partial z_1^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_i^{(l)}}{\partial w_{ij}^{(l)}}, \dots, \frac{\partial z_{m^{(l)}}^{(l)}}{\partial w_{ij}^{(l)}} \right] \\ &= \left[0, \dots, \frac{\partial (\mathbf{w}_{i:}^{(l)} \mathbf{a}^{(l-1)} + b_i^{(l)})}{\partial w_{ij}^{(l)}}, \dots, 0 \right] \\ &= \left[0, \dots, a_j^{(l-1)}, \dots, 0 \right] \\ &\triangleq \mathbb{I}_i(a_j^{(l-1)}) \in \mathbb{R}^{m^{(l)}},\end{aligned}$$

计算对偏置项的偏导数：

$$\frac{\partial \mathbf{z}^{(l)}}{\partial \mathbf{b}^{(l)}} = \mathbf{I}_{m^{(l)}} \in \mathbb{R}^{m^{(l)} \times m^{(l)}}$$

$$\mathbf{z}^{(l+1)} = W^{(l+1)} \mathbf{a}^{(l)} + \mathbf{b}^{(l+1)}$$

$$\mathbf{a}^{(l)} = f_l(\mathbf{z}^{(l)})$$

$$\frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} = (W^{(l+1)})^T$$

计算误差项

$$\frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} = \frac{\partial f_l(\mathbf{z}^{(l)})}{\partial \mathbf{z}^{(l)}}$$

$$= \text{diag}(f'_l(\mathbf{z}^{(l)}))$$

$$\delta^{(l)} \triangleq \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l)}}$$

$$\begin{aligned} &= \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l+1)}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{z}^{(l+1)}} \\ &= \text{diag}(f'_l(\mathbf{z}^{(l)})) \cdot (W^{(l+1)})^T \cdot \delta^{(l+1)} \\ &= f'_l(\mathbf{z}^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)}), \end{aligned}$$

第 l 层的误差项可以通过第 $l+1$ 层的误差项计算得到

Hadamard积运算符，表示每个元素相乘。

反向传播

在计算出上面三个偏导数之后, 公式 (4.49) 可以写为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial w_{ij}^{(l)}} = \mathbb{I}_i(a_j^{(l-1)})\delta^{(l)} = \delta_i^{(l)} a_j^{(l-1)}.$$

进一步, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层权重 $W^{(l)}$ 的梯度为

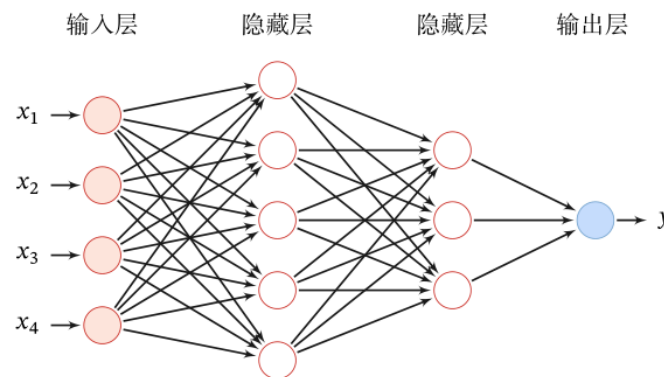
$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial W^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top.$$

同理, $\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$ 关于第 l 层偏置 $\mathbf{b}^{(l)}$ 的梯度为

$$\frac{\partial \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}.$$

在计算出每一层的误差项之后，就可以得到每一层的梯度。因此，使用误差反向传播算法的前馈神经网络训练过程可以分为以下三步：

- (1) 前馈计算每一层的净输入 $\mathbf{z}^{(l)}$ 和激活值 $\mathbf{a}^{(l)}$ ，直到最后一层；
- (2) 反向传播计算每一层的误差项 $\delta^{(l)}$ ；
- (3) 计算每一层参数的偏导数，并更新参数。



算法 4.1 使用反向传播算法的随机梯度下降训练过程

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α , 正则化系数 λ , 网络层数 L , 神经元数量 $M_l, 1 \leq l \leq L$.

```
1 随机初始化  $\mathbf{W}, \mathbf{b}$ ;  
2 repeat  
3   对训练集  $\mathcal{D}$  中的样本随机重排序;  
4   for  $n = 1 \cdots N$  do  
5     从训练集  $\mathcal{D}$  中选取样本  $(\mathbf{x}^{(n)}, y^{(n)})$ ;  
6     前馈计算每一层的净输入  $\mathbf{z}^{(l)}$  和激活值  $\mathbf{a}^{(l)}$ , 直到最后一层;  
7     反向传播计算每一层的误差  $\delta^{(l)}$ ; // 公式 (4.63)  
      // 计算每一层参数的导数  
8      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^\top$ ; // 公式 (4.68)  
9      $\forall l, \quad \frac{\partial \mathcal{L}(\mathbf{y}^{(n)}, \hat{\mathbf{y}}^{(n)})}{\partial \mathbf{b}^{(l)}} = \delta^{(l)}$ ; // 公式 (4.69)  
      // 更新参数  
10     $\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \alpha (\delta^{(l)} (\mathbf{a}^{(l-1)})^\top + \lambda \mathbf{W}^{(l)})$ ;  
11     $\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \alpha \delta^{(l)}$ ;  
12  end  
13 until 神经网络模型在验证集  $\mathcal{V}$  上的错误率不再下降;  
    输出:  $\mathbf{W}, \mathbf{b}$ 
```

计算图与自动微分

◆ 手动微分

写好表达式，计算机计算

◆ 数值微分

差分 Δx 误差、计算复杂度

◆ 符号微分

自动计算表达式 复杂、经验性

◆ 自动微分

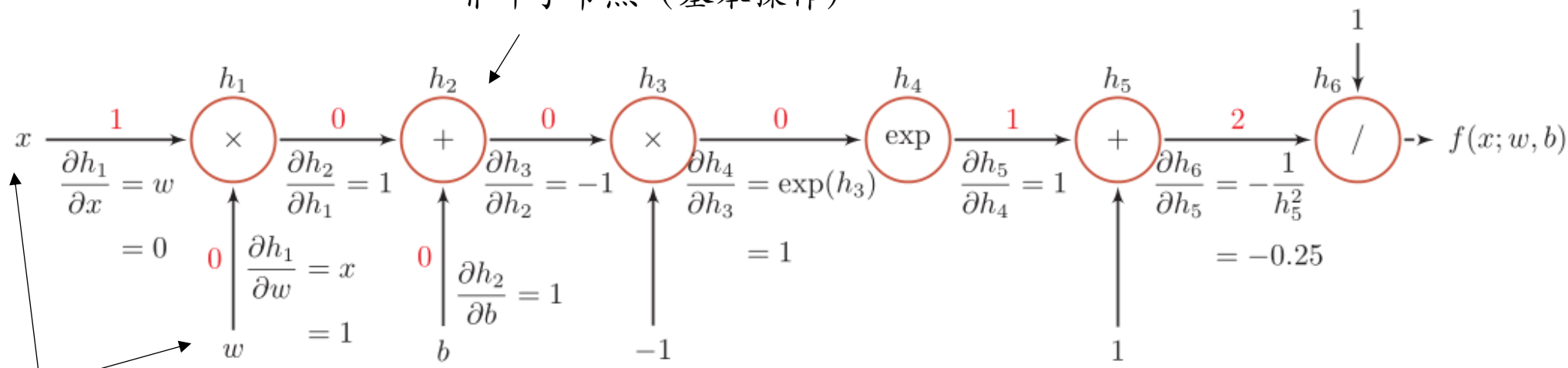
- ◆自动微分 (Automatic Differentiation, AD) 是一种可以对一个函数或一段程序进行计算导数的方法。利用链式法则来自动计算一个复合函数的梯度。
- ◆计算图: 将复合函数分解为一系列的基本操作, 就构成了一个计算图。计算图是数学运算的图形化表示。计算图中的每个非叶子节点表示一个基本操作, 每个叶子节点为一个输入变量或常量。

◆ 例：

◆ 一个神经网络中常见的复合函数

$$f(x; w, b) = \frac{1}{\exp(-(wx + b)) + 1}.$$

◆ 计算图如下：



叶子节点（输入变量或常量）

◆ 例：

◆ 整个复合函数 $f(x; w, b)$ 关于参数 w 和 b 的导数可以通过计算图上的节点与参数之间路径上所有的导数连乘来得到：

函数	导数	导数
$h_1 = x \times w$	$\frac{\partial h_1}{\partial w} = x$	$\frac{\partial h_1}{\partial x} = w$
$h_2 = h_1 + b$	$\frac{\partial h_2}{\partial h_1} = 1$	$\frac{\partial h_2}{\partial b} = 1$
$h_3 = h_2 \times -1$	$\frac{\partial h_3}{\partial h_2} = -1$	
$h_4 = \exp(h_3)$	$\frac{\partial h_4}{\partial h_3} = \exp(h_3)$	
$h_5 = h_4 + 1$	$\frac{\partial h_5}{\partial h_4} = 1$	
$h_6 = 1/h_5$	$\frac{\partial h_6}{\partial h_5} = -\frac{1}{h_5^2}$	

$$\frac{\partial f(x; w, b)}{\partial w} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w},$$

$$\frac{\partial f(x; w, b)}{\partial b} = \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial b}.$$

◆ 令 $x=1, w=0, b=0$, 可以得到

基本函数的导数十分简单

$$\begin{aligned} \frac{\partial f(x; w, b)}{\partial w} \Big|_{x=1, w=0, b=0} &= \frac{\partial f(x; w, b)}{\partial h_6} \frac{\partial h_6}{\partial h_5} \frac{\partial h_5}{\partial h_4} \frac{\partial h_4}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w} \\ &= 1 \times -0.25 \times 1 \times 1 \times -1 \times 1 \times 1 \\ &= 0.25. \end{aligned}$$

◆前向模式和反向模式

反向模式和反向传播的计算梯度的方式相同

$N \gg M$, 使用反向模式

$M \gg N$, 使用前向模式

◆如果目标函数和参数之间有多条路径，可以将这多条路径上的导数再进行相加，得到最终的梯度。

◆前馈神经网络的训练过程可以分为以下三步

1. 前向计算每一层的状态和激活值，直到最后一层
2. 反向计算每一层的参数的偏导数
3. 更新参数

◆静态计算图：在编译时构建计算图，计算图构建好之后在程序运行时不能改变。

代表框架: Theano、Tensorflow

◆动态计算图：在程序运行时动态构建。

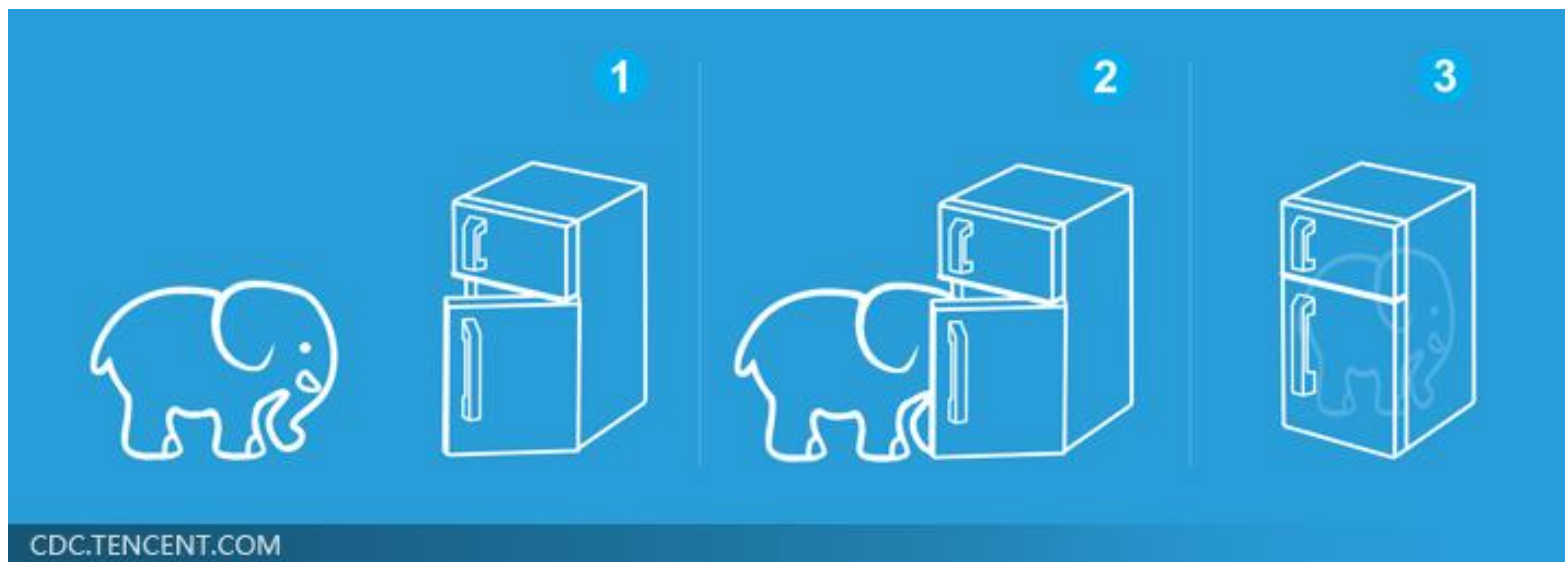
代表框架: Chainer、PyTorch

◆两种构建方式各有优缺点。

◆静态计算图在构建时可以进行优化，并行能力强，但灵活性比较低。

动态计算图则不容易优化，当不同输入的网络结构不一致时，难以并行计算，但是灵活性比较高。

深度学习的三步走

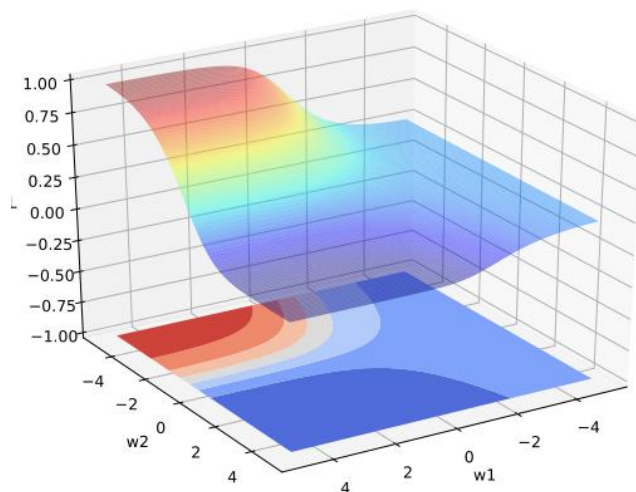


优化问题

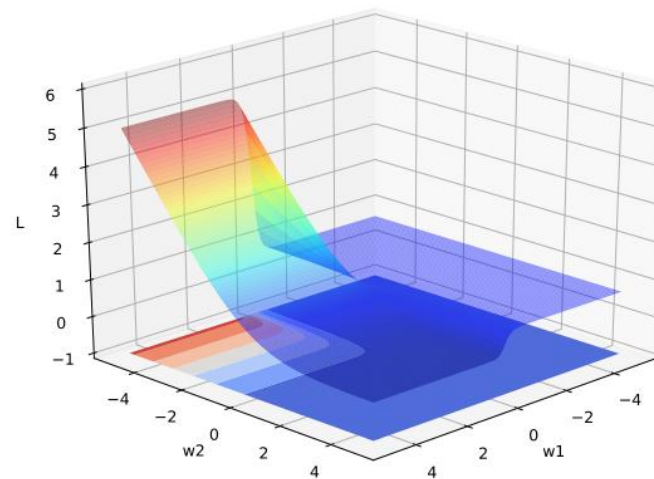
◆ 非凸优化问题

◆ 以一个简单的神经网络为例，当选择神经网络为 $y = \sigma(w_2\sigma(w_1x))$ 时，如图所示，两种损失函数都是关于参数的非凸函数。

局部最优
鞍点



(a) 平方误差损失



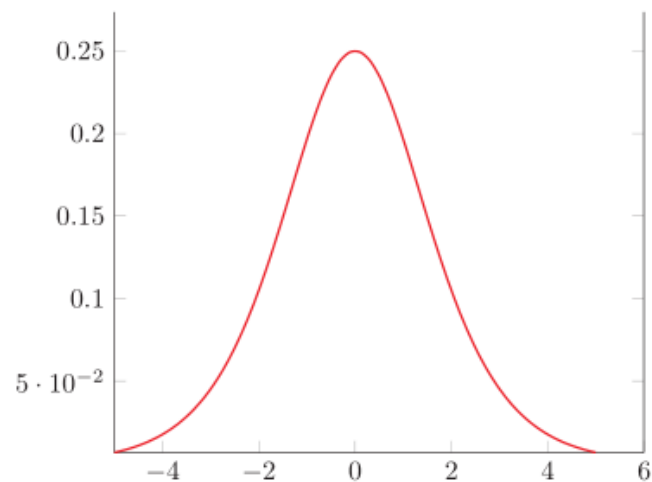
(b) 交叉熵损失

图 4.9 神经网络 $y = \sigma(w_2\sigma(w_1x))$ 的损失函数

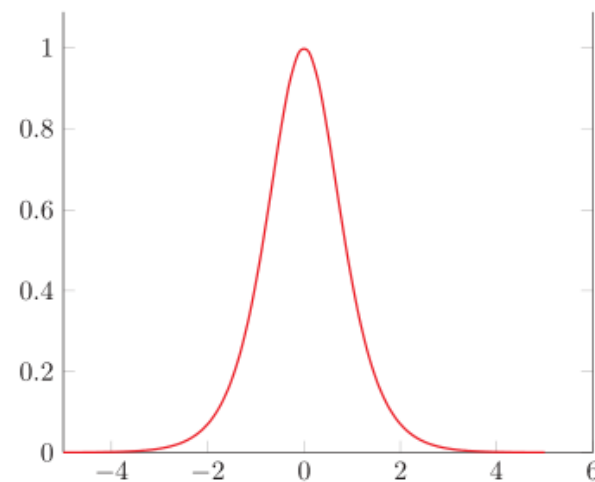
◆ 梯度消失问题:

◆ 误差从输出层反向传播时，在每一层都要乘以该层的激活函数的导数。以Sigmoid型函数为例，Sigmoid型函数的导数的值都小于或等于1。当使用Sigmoid型函数时，误差经过每一层传递都会不断衰减。当网络层数很深时，梯度就会不停衰减，甚至消失，使得整个网络很难训练。

梯度在 1 附近
ReLU



(a) logistic 函数的导数



(b) tanh 函数的导数

◆需求

- ◆计算资源要大
- ◆数据要多
- ◆算法效率要好：即收敛快

◆难点

- ◆参数过多，影响训练
- ◆非凸优化问题：即存在局部最优而非全局最优解，影响迭代
- ◆梯度消失问题，下层参数比较难调
- ◆参数解释起来比较困难

◆ 知识点

- ◆ 激活函数
- ◆ 误差反向传播
- ◆ 自动微分与计算图

◆ 作业

课后题：4-2、4-5

◆ 作业提交：

- ◆ 各位同学将理论课作业（pdf格式）和实验课作业（ipynb格式）合并为一个压缩包，文件命名“学号_姓名”
- ◆ 各班班长收齐并制成班级作业的压缩包，发到邮箱：nuoxu25@mails.jlu.edu.cn（许诺）

◆ 截至时间：

◆ 下周日晚9点

- ◆ 若在下周日9点后，下周一晚9点前提交，则视为晚交，此次作业按60%分数计算
- ◆ 若在下周一晚9点后提交，则不计分